

Data Mining

Lab - 10

¥@\$# Kakadiya | 23010101121 |
20/8/2025

Implement Decision Tree(ID3) in python

Uses Information Gain to choose the best feature to split.

Recursively builds the tree until stopping conditions are met.

1. Calculate Entropy for the dataset.
2. Calculate Information Gain for each feature.
3. Choose the feature with maximum Information Gain.
4. Split dataset into subsets for that feature.
5. Repeat recursively until:

All samples in a node have the same label.

No features are left.

No data is left.

Step 2. Import the dataset from this [address](#).

import Pandas, Numpy

```
In [1]: import pandas as pd  
import numpy as np
```

Create Following Data

```
In [2]: data = pd.DataFrame({  
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny', 'Sunny',  
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild',  
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High', 'Normal',  
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak', 'Weak',  
    'PlayTennis': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes']  
})
```

In [3]: data

Out[3]:

	Outlook	Temperature	Humidity	Wind	PlayTennis
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Now Define Function to Calculate Entropy

```
In [4]: def entropy(target_col):
        elements, counts = np.unique(target_col, return_counts=True)
        entropy = -np.sum([(counts[i]/np.sum(counts))*np.log2(counts[i]/np.sum(counts)) for i in range(len(counts))])
        return entropy
```

Testing of Above Function -

y = np.array(['Yes', 'No', 'Yes', 'Yes'])

Function Call - > entropy(y)

output - 0.8112781244591328

```
In [5]: y = np.array(['Yes', 'No', 'Yes', 'Yes'])
        print(entropy(y))
```

0.8112781244591328

Define function to Calculate Information Gain

```
In [6]: def information_gain(data, split_attribute_name, target_name):
        """
        Calculate the information gain of a dataset for a given attribute.
        """
        # Calculate the entropy of the whole dataset
        total_entropy = entropy(data[target_name])

        # Calculate the entropy of the dataset after splitting by the attribute
        values, counts = np.unique(data[split_attribute_name], return_counts=True)
        weighted_entropy = np.sum([(counts[i]/np.sum(counts))*entropy(data.where(data[split_attribute_name] == values[i])) for i in range(len(values))])

        return total_entropy - weighted_entropy
```

```
# Calculate the information gain
information_gain = total_entropy - weighted_entropy
return information_gain
```

Testing of Above Function-

```
data = pd.DataFrame({ 'Weather': ['Sunny', 'Sunny', 'Rain', 'Rain'], 'Play': ['Yes', 'No', 'Yes', 'Yes'] })
```

Function Call - > information_gain(data, 'Weather', 'Play')

Output - 0.31127812445913283

```
In [7]: data_test = pd.DataFrame({
'Weather': ['Sunny', 'Sunny', 'Rain', 'Rain'],
'Play':    ['Yes', 'No',    'Yes', 'Yes']
})

print(information_gain(data_test, 'Weather', 'Play'))
```

0.31127812445913283

Implement ID3 Algo

```
In [8]: def id3(data, features, target="PlayTennis"):
# If all labels are same → return the label
if len(np.unique(data[target])) == 1:
    return np.unique(data[target])[0]

# If no features left → return majority label
elif len(features) == 0:
    return np.unique(data[target])[np.argmax(np.unique(data[target], return_counts=True)[1])]

else:
    # Choose best feature
    item_values = [information_gain(data, feature, target) for feature in features]
    best_feature_index = np.argmax(item_values)
    best_feature = features[best_feature_index]

    # Create tree structure
    tree = {best_feature: {}}

    # Remove best feature from feature list
    features = [i for i in features if i != best_feature]

    # For each value of best feature → branch
    for value in np.unique(data[best_feature]):
        sub_data = data.where(data[best_feature] == value).dropna()
        subtree = id3(sub_data, features, target)
        tree[best_feature][value] = subtree

    return tree
```

Use ID3

```
In [9]: features = data.columns[:-1] # Exclude the target column
tree = id3(data, features)
```

Print Tree

```
In [10]: print(tree)
```

```
{'Outlook': {'Overcast': 'Yes', 'Rain': {'Wind': {'Strong': 'No', 'Weak': 'Yes'}}}, 'Sunny': {'Humidity': {'High': 'No', 'Normal': 'Yes'}}}}
```

Extra: Create Predict Function

```
In [11]: def predict(tree, sample):
          # If the node is a leaf node, return the label
          if not isinstance(tree, dict):
              return tree

          # Get the attribute to split on
          attribute = list(tree.keys())[0]
          attribute_value = sample[attribute]

          # Traverse the tree based on the attribute value
          if attribute_value in tree[attribute]:
              return predict(tree[attribute][attribute_value], sample)
          else:
              # Handle cases where the attribute value is not in the tree (e.g., return majority class
              # For simplicity, this implementation will return None or raise an error.
              # A more robust implementation would handle unknown values.
              return None # Or raise an error
```

Extra: Predict for a sample

sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Your Answer ?

```
In [12]: sample = {'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}
```

```
In [13]: prediction = predict(tree, sample)
          print("Prediction:", prediction)
```

Prediction: No

Task

Complete the Python notebook to implement and test the ID3 decision tree algorithm, including functions for entropy, information gain, ID3 tree building, and prediction.

Complete entropy function

Subtask:

Implement the calculation of entropy in the provided entropy function.

Reasoning: Implement the entropy function based on the provided instructions to calculate the entropy of a given target variable.

Complete information_gain function

Subtask:

Implement the calculation of information gain in the provided information_gain function.